

09：第二版确定采用的双向 NBFNet

1. 先说明当前事实

本章描述的是已经冻结的 NBFNet 主模型设计，不是当前已经完成的代码。

截至 2026-07-29，统一数据接口和 CUDA 无传播 MLP 已经完成，但 NBFNet 主模型尚未实现。

因此不能说“第二版 NBFNet 已训练”或“传播已证明有效”；当前成立的是数据、标签和无传播学习基线，下一阶段才验证图传播的独立贡献。

2. 先理解 NBFNet 在本项目中的含义

Neural Bellman-Ford Network (NBFNet) 可以理解成“让 Bellman-Ford 式的多跳候选传播变得可学习”。它不是把经典最短路算法改成近似神经网络，也不会替代在线 Dijkstra。本项目只借用它的三个思想：从指定需求源注入状态、沿有向边逐轮传播、让不同道路和传播深度的影响由真实区域收益来决定。

原始 NBFNet 通常一次处理一个查询条件。Porto 图和历史 OD 数量都很大，第二版不会逐条 OD 在全图上重复计算，而是把历史 OD 压缩成少量“需求原型”，对原型分批传播，再按原型出现频率汇总区域价值。若去掉这些条件而只保留两个节点频率场，它就只是普通 GNN，不能作为第二版主模型。

3. 模型要接收什么输入

对每个道路节点，第一轮核心输入计划包含：

- 历史窗口中作为起点的归一化次数；
- 历史窗口中作为终点的归一化次数；
- 入度；
- 出度；

- 相邻边长度；
- 道路类型。

此外，模型还接收由历史 OD 形成的需求原型条件。原型至少保留起点集合、终点集合和出现权重；原型构造只使用历史 OD 和静态道路图，不使用历史最短路径。

入度与出度

- 入度：有多少条有向边指向该节点；
- 出度：有多少条有向边从该节点出发。

它们描述道路连接角色，例如一个节点可能是多条路汇入的点，或有多个可驶出方向。

坐标不是第一轮默认核心特征，只计划作为单独消融。这样能检查模型是否只是记住固定城市位置，而没有学习可迁移的道路结构。

4. 什么是节点表示

模型不能直接对“节点编号 12345”做有意义计算。它会把每个节点的原始特征转换成一个数值向量，例如：

```
[0.12, -0.38, 0.91, ...]
```

这个向量叫作表示、嵌入或 hidden state。每一维不一定有人类预先指定的含义，而是训练中为了预测区域收益逐渐形成。

5. 两套初始需求场

对每个需求原型，每个节点建立两套独立状态：

- O^0 ：Origin 初始场，强调该原型的起点需求；
- D^0 ：Destination 初始场，强调该原型的终点需求。

起点编码器 f_O 和终点编码器 f_D 不共享参数，因为“从这里出发的需求”与“要到这里

结束的需求”在有向路网上语义不同。

公式：

$$O_v^{(0)} = f_O(\text{节点静态特征}, \text{节点 } v \text{ 的起点计数})$$

$$D_v^{(0)} = f_D(\text{节点静态特征}, \text{节点 } v \text{ 的终点计数})$$

上标 (0) 表示传播前的第 0 层。

6. 为什么起点场正向传播

历史中经常从节点 u 出发的需求，可能沿 u 的出边影响下游道路。起点场因此按原图方向：

$$u \rightarrow v$$

传播到 v 。

节点 v 从所有指向自己的入边发送者接收起点消息。

7. 为什么终点场反向传播

如果许多需求最终到达节点 t ，那么能够沿有向道路通往 t 的上游区域可能有价值。

要从终点向“可能到达它的来源方向”传播，就要沿反图前进。等价地，节点 v 从它在原图中的出边邻居接收终点方向消息。

起点和终点方向分开，正是第二版相对第一版固定无向扩散的关键区别。

8. 什么是消息

在一层图传播中，相邻节点不会原样复制所有状态，而是经过一个可学习变换 Φ 产生消息。

消息可以同时读取：

- 发送节点状态；

- 接收节点状态；
- 边长；
- 道路类型；
- 方向。

这样模型可以学到“不同道路类型和距离上的需求影响应该怎样衰减或变形”。

9. 什么是注意力

一个节点可能有多个邻居，但不同邻居并不一定同样重要。注意力机制为每条消息计算权重 α ，再做加权求和。

起点场的一层可概念化为：

候选新起点状态
= 所有入边邻居消息 $\times$ 各自注意力权重 的总和

注意力权重由训练学习，不是预先规定“短边一定更重要”。

10. 什么是门控

如果每一层都完全替换节点状态，多层传播可能把原始本地需求冲淡。门控 g 决定保留多少新消息、多少旧状态：

新状态 = $g \times$ 聚合消息 + $(1 - g) \times$ 旧状态

g 接近 1 时更相信邻居传播；接近 0 时更多保留本地信息。它可以是每个节点、每层、每个向量维度不同的可学习值。

11. 为什么两套传播参数不能共享

如果起点和终点传播完全共享参数，模型被迫假设：

需求从起点向外扩散的规律，与从终点沿反图追溯的规律相同。

道路方向、出入口结构和 OD 分布未必满足这个对称假设。第二版完整模型因此使用两套参数，而“共享参数”会作为消融对照。

12. 什么是传播深度

一层消息传递让节点看到一跳邻居，两层可以间接看到两跳邻居。层数越深，理论感受范围越大，但也可能：

- 计算更贵；
- 把局部差异过度平滑；
- 引入无关远处需求；
- 训练更不稳定。

第一版手工选择固定扩散步数。第二版计划同时保留第 0 到第 L 层状态，并学习融合权重。

13. 可学习深度融合

模型为不同层学习 $\beta_0, \beta_1, \dots, \beta_L$ ，经过 softmax 归一化后加权组合。

Softmax 把任意实数变成总和为 1 的非负权重。例如原始打分经过 softmax 后可能得到：

```
第 0 层: 0.10
第 1 层: 0.45
第 2 层: 0.30
第 3 层: 0.15
```

这表示模型更依赖一跳和两跳信息，但仍保留本地与更远信息。权重也可作为解释结果报告。

后续改进一：倍增传播

普通逐层传播通常每增加一层只多看到一跳道路邻居。道路路径可能包含很多节点，直接堆叠大量层会很慢。基础 NBFNet 验证成立后，第二版将尝试按 1、2、4、8... 的尺度传播，

让较少层数覆盖更远道路范围。

倍增连接只能从静态道路图本身构造，例如重复邻接或先把相邻道路聚合成粗粒度节点。它不允许读取历史最短路径，也不能根据标签窗口或最终测试窗口决定跳到哪里。

后续改进二：跨层注意力残差

深层传播可能逐渐冲淡最初的本地需求。跨层注意力残差让当前层有选择地读取第 0 层、较浅层和较深层状态，再通过残差通道保留原始信息。这样区域读出可以同时使用本地入口结构、中程需求和远程需求。

这项设计借鉴深层注意力网络中的跨层信息保留思想。正式实验不把它和倍增传播一次性混入基础模型，而是比较：基础 NBFNet、只加倍增传播、只加跨层注意力残差、两者同时加入。

14. 起终点场怎样组合

每一层计划组合四类信息：

1. 起点状态 O ；
2. 终点状态 D ；
3. 逐元素乘积 $O \odot D$ ；
4. 逐元素绝对差 $|O - D|$ 。

逐元素乘积

两个同长度向量在相同位置相乘。它用来表达起点影响和终点影响同时较强的潜在交汇。

绝对差

它描述两个方向需求的不平衡。例如一个区域可能大量向外出发，却很少作为终点方向的上游。

这些只是可学习表示，不是声称模型恢复了真实最短路径走廊。

15. 从节点表示到区域表示：池化

一个候选包含 512 个节点，但需要输出一个区域分数。把一组节点向量汇总为一个区域向量叫作池化。

第二版不只做普通节点池化，还要显式区分区域内部和边界：

- mean pooling：512 个节点表示取平均；
- max pooling：每一维取所有节点中的最大值；
- attention pooling：学习哪些节点对区域价值更重要，再加权汇总。
- 入口边界：需求从区域外进入区域的潜在强度；
- 出口边界：需求从区域离开的潜在强度；
- 内部交汇：起点场和终点场在区域内部的兼容程度。

这样读出层才与 shortcut 的“入口—内部—出口”工作机制对应，而不是只选择需求最热的节点集合。

池化应对节点排列顺序不敏感，因为同一个节点集合换个列表顺序仍是同一区域。

16. 需求分支与成本分支

计划结构：

```
demand_branch = 双向 NBFNet 区域池化
cost_branch   = 边界数 + 内部节点数 + shortcut 数
prediction    = 两个分支融合后的预测头
```

预测头是把区域向量转换成一个标量预测收益 $\hat{y}_R$ 的小型神经网络。

核心传播分支禁止读取局部接入工作量、查询耗时、回退率等容易复制答案的统计。

17. 损失函数是什么

损失函数把“预测有多错”压缩成一个数。训练器计算损失对参数的梯度，再调整参数使损失减小。

第二版计划总损失由三部分组成。

Huber 损失

它用于学习具体收益数值。误差小时像平方误差，误差很大时增长更温和，减少极端候选对训练的支配。

Pairwise ranking 损失

它抽取两个候选，要求真实收益更高的候选预测分数也更高。项目最终更关心选出 Top-K，所以只拟合数值还不够，排序损失能直接强调相对顺序。

未来需求辅助损失

可以让节点表示小权重预测后续起终点需求，帮助表示学习。但它只能是辅助任务，不能替代真实区域收益主损失，否则又回到“优化需求 Proxy，而非查询收益”。

18. 什么是反向传播

一次训练步骤可简化为：

输入历史特征

↓ 前向计算

预测区域收益

↓ 与真实标签比较

得到损失

↓ 反向计算每个参数对损失的影响

更新预测头、池化、深度权重、门控、注意力和编码器

“端到端”表示损失梯度能够一直传回最早的起点场与终点场，而不是只训练最后一个预测头。

19. 为什么先完成无传播 MLP

MLP 是多层感知机。这里它可以读取区域中节点的原始历史频率和静态特征，但不在图边上传递邻居消息。

先做 MLP 有三层意义：

1. 验证真实标签是否能被历史输入学到；
2. 建立不使用图传播的强学习基线；
3. 检查数据、训练和区域池化链路，再增加复杂传播模块。

如果 MLP 都无法稳定学习标签，直接训练更复杂 GNN 很难定位问题。

当前这一步已经完成。统一接口把历史需求和静态道路特征池化为 64 维区域输入，使用 64、32 两个隐藏层训练五个随机种子。Candidate holdout 上：

- Spearman: 0.8914 ± 0.0238 ，原始频率为 0.7957 ；
- NDCG@K: 0.9556 ± 0.0153 ，原始频率为 0.9661 ；
- Top-K 平均真实收益: 126.000 ± 1.853 ，原始频率为 128.530 。

这五次正式训练均由 PyTorch CUDA 在 NVIDIA GeForce RTX 3050 Laptop GPU 上完成。

五个种子全部通过可学习性阶段门，状态为 `ready_for_nbfnet`。这里的 holdout 是同一 H→Y 时间对内留出的候选，不是最终未来时间测试；它证明数据与无传播学习链路成立，但不证明跨时间泛化。

20. 必做对比

第二版计划至少比较：

1. 固定候选上的随机排序；
2. 原始起终点频率排序；
3. Risk-only；

4. 第一版 Proxy-only;
5. 原始起终点频率 + 区域 MLP;
6. 第一版固定无向扩散 + MLP;
7. 普通共享参数 GraphSAGE;
8. 第二版 OD 条件化双向 NBFNet;
9. 训练期真实收益 Oracle 上界。

21. 必做消融

包括：

- 删除全部传播;
- 换成固定均值扩散;
- 只保留起点场;
- 只保留终点场;
- 两场共享参数;
- 忽略道路方向;
- 删除边长与道路类型;
- 删除交汇项;
- 只用最后一层，不学习深度融合;
- 删除排序损失。
- 打乱起点—终点配对，只保留边际频率;
- 用普通 GraphSAGE 替换 NBFNet 式逐轮传播;
- 删除需求原型条件，只保留全局起点和终点场;
- 删除入口—出口边界读出，只做区域平均池化。
- 删除倍增传播，恢复普通逐层邻接传播;
- 删除跨层注意力残差，只保留相邻层更新;

- 分别比较基础、倍增、跨层残差和组合模型，并匹配参数量与训练预算。

22. 什么结果才算 NBFNet 传播成立

最低证据要求是：

- OD 条件化双向 NBFNet 稳定优于原始频率、固定扩散、GraphSAGE 和无传播 MLP；
- 删除起点场或终点场都会稳定变差；
- 有向传播优于无向传播；
- 保留 OD 配对关系优于打乱配对的边际频率模型；
- 边界感知读出优于普通区域池化；
- 提升不只出现在训练损失，还出现在未来窗口的真实工作量和在线配对耗时；
- 多个随机种子结论一致；
- 所有距离正确率仍为 100%。

如果完整模型只优于随机，却没有优于 MLP 和 cost-only，就不能说双向需求传播贡献成立。